

UAS E2EE Data Protection Lite Application Developer Guide - 6.0.1-GA

Table of Contents

1. Preface - UAS E2EE Data Protection Lite Application Developer Guide.....	3
2. Introduction to UAS E2EE Data Protection Lite Application Development....	4
3. UAS E2EE Data Protection Lite Application Integration.....	6
4. UAS E2EE Data Protection Lite Libraries.....	15

1. Preface - UAS E2EE Data Protection Lite Application Developer Guide

The AccessMatrix Universal Authentication (UAS) Server provides End-to-End Encryption (E2EE) Data Protection (DP) capabilities. This document provides detailed information on UAS E2EE DP Lite, as well as important implementation and integration details.

Intended Reader

This guide is intended for experienced application developers and security administrators.

Document Scope

This document serves as an application integration guide for using AccessMatrix Universal Authentication Server's End-to-End Encryption (E2EE) for Data Protection (DP) feature.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

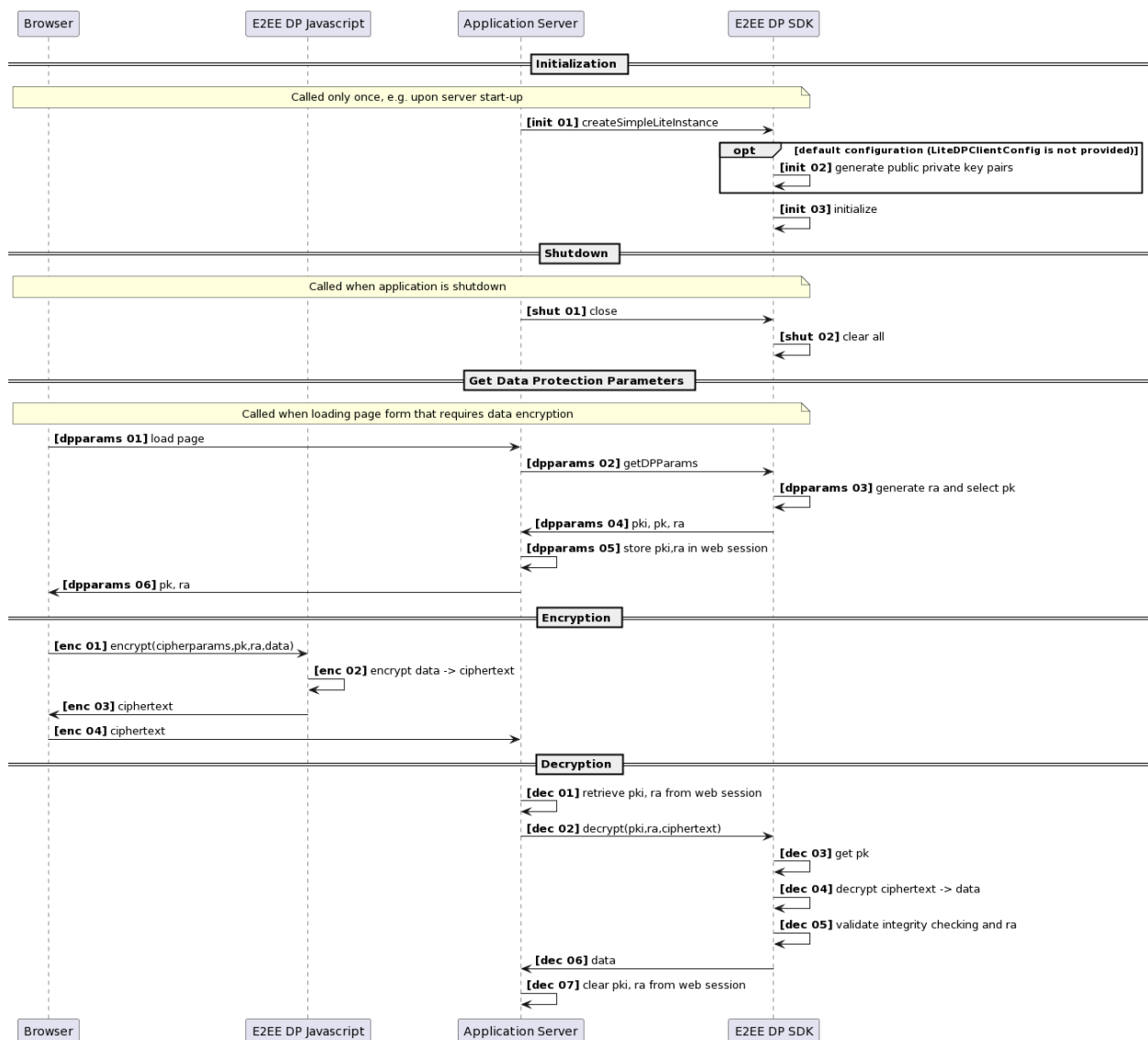
For more information, see the following documents:

- *AccessMatrix XML-RPC and SOAP API Reference*

2. Introduction to UAS E2EE Data Protection Lite Application Development

End-to-End Encryption for Data Protection (E2EE DP) provides confidentiality of data from the client like the browser to the other end like application. Secure Socket Layer (SSL) ends at a web server. Sensitive information may get exposed in the internal network between the web server and application server. In addition to confidentiality, E2EE DP also provides data integrity checking and replay attack mitigation.

The E2EE process is depicted in the diagram below:



Symmetric Encryption and Integrity Checking Configuration

- **RSA Encryption and Hash**
 - Encryption for **short data** (data length is limited by RSA key size)

-
- Integrity checking is using the hash algorithm, such as SHA2-256

- **RSA Encryption, Symmetric Encryption, and Hash**

- Encryption for **long data** (limited by memory usage)
 - Symmetric encryption is added using the symmetric algorithm, such as AES
 - Integrity checking is using the hash algorithm, such as SHA2-256
-

3. UAS E2EE Data Protection Lite Application Integration

E2EE Integration With Java Application

E2EE DP Setup

DataProtSvc needs to be created via the DataProtSvcFactory method. Once instantiated, DataProtSvc has two important methods: getDPPParams and decrypt.

The required libraries and Javadoc are packaged in *WEB-INF/lib/* directory of **e2eedp.war** (sample application) of AccessMatrix server's SDK.

Usage

Initialization

The DataProtSvc needs to be created/initialized once. The application needs to keep referencing to the DataProtSvc instance throughout the lifetime of the DataProtSvc.

Java

```
DataProtSvc dp =  
DataProtSvcFactory.createSimpleLiteInstance(Integer.parseInt(numOfKeyPair  
s.trim()));
```

The E2EE DataProtSvc will be initialized with integrity checking (SHA2-256) enabled by default.

Shutdown

DataProtSvc needs to be closed when the application does not need it anymore or before the application is shutdown.

Java

```
dp.close();
```

Getting Data Protection Parameters

DataProtSvc.getDPPParams needs to be called when the user loads the page that will submit user data such as transaction page. This is to get the public key, public key index, and a random number. The

application needs to provide the public key and random number to the client side library. These pieces of information will be used to encrypt sensitive data. In addition, the application also needs to remember the public key index and a random number. This can be done by saving the web application session. These pieces of information need to be fetched and provided when calling `DataProtSvc.decrypt`.

Java

```
DataProtSvc dp = (DataProtSvc)
application.getAttribute("DataProtection");
DPPParams dpParms = dp.getDPPParams(null, null); // proxy
and moduleId can be set to NULL for E2EE DP Lite
System.out.println("+ public key index:
"+dpParms.getPki());
System.out.println("+ public key: "+dpParms.getPk());
System.out.println("+ random number:
"+dpParms.getRandomNumber());

session.setAttribute("pki", dpParms.getPki());
session.setAttribute("rn", dpParms.getRandomNumber());
```

Decryption

Public key index and the random number were given when calling `getDPPParams` need to be provided. After the decryption, the public key index and the random number need to be removed (e.g. removed from the user's web session).

Java

```
DataProtSvc dp = (DataProtSvc)
application.getAttribute("DataProtection");
String ciphertext = request.getParameter("ciphertext");
Integer pki = (Integer) session.getAttribute("pki");
String rn = (String) session.getAttribute("rn");
if (pki == null || rn == null)
    message = "Decryption failed because there is no
existing data protection session (pki and random number
are missing)";
else {
    PlainText plaintext = dp.decrypt(null, null,
ciphertext, pki, rn);
    String data = plaintext.getAsString();
    session.removeAttribute("pki");
    session.removeAttribute("rn");
}
```

E2EE Integration With .NET Application

E2EE DP Setup

The following Dynamic-link libraries (DLL) are needed:

- **e2eedplite-net.dll** (i-Sprint e2eedplite .NET wrapper)
- **e2eedplite.dll** (i-Sprint e2eedplite C/C++ library)
- **libeay32.dll** (openssl, <https://www.openssl.org/>, BSD style license suitable for commercial use)

To set up:

1. Import **e2eedplite-net.dll** into your .NET project.
2. Place **e2eedplite.dll** and **libeay32.dll** into the same folder as **e2eedplite-net.dll**.



Note: Binaries are available for both 32-bit and 64-bit windows.

Usage

The following sections contains C# code snippets. To use, include the following directive:

C#

```
using com.isprint.am.dp.api;
```

Initialization

Initialization for short data using RSA encryption and hashing.

C#

```
int numKeys=1; //min numKeys=1, max numKeys=4
int useHash=1;

//For .NET4.0
e2eedplite.Init(numKeys, useHash,
SymmetricAlgo.NotUsingSymmetricEncryption);

//For .NET3.5
e2eedplite.Init(numKeys, useHash,
SymmetricAlgo.NotUsingSymmetricEncryption,
    null, null, null, null, null, null, null, null, null,
null, null, null);
```

Initialization for long data using RSA and AES-128 encryption and hashing.

C#

```
int numKeys=1; //min numKeys=1, max numKeys=4
int useHash=1;

//For .NET4.0
e2eedplite.Init(numKeys, useHash, SymmetricAlgo.AES_128);

//For .NET3.5
e2eedplite.Init(numKeys, useHash, SymmetricAlgo.AES_128,
    null, null, null, null, null, null, null, null, null, null, null);
```

Initialization for long data using RSA and AES-256 encryption and hashing.

C#

```
int numKeys=1; //min numKeys=1, max numKeys=4
int useHash=1;

//For .NET4.0
e2eedplite.Init(numKeys, useHash, SymmetricAlgo.AES_256);

//For .NET3.5
e2eedplite.Init(numKeys, useHash, SymmetricAlgo.AES_256,
    null, null, null, null, null, null, null, null, null, null, null);
```

Initialization with user defined keypair

C#

```
const string RSA_KEY_N =
"E3333C6ED2B525BA9528324A0F875C839EC09858668EAF3B132DC2F17B86BDBC330269F92EA
21CFA68BAE4EC7C87DBA2
7BAF8E1BC9E30386D996E96ED037F62EC908B1BD6FF800330E4C7F8FD1EF051C64C5E398E77E
6B754E9C8016A659E4A01
936DC2F94CC539C05E004E6D2210D782618E12F391E5ED2A97824839E3D024E0AFE6090D8EBF
B2A8FBBCCCE4398FF4CE3C
B4DD1A537FFF10DEBE01F4E9969C0F3D340B6F340F795C88B41A7595F1AF5D212660E610F88D
39F2F64DA8148283DF744
6135A0B26E20D6E7D06E9938A5FB2458C10A21851C085E16B2592766C6999D670D3B0912641F
44D06748D152B6AD12FF4
2D196B97BEA828239746045A1EC7";
const string RSA_KEY_E = "010001";
const string RSA_KEY_D =
"3CF3CBECB75E9DC696A9780462722021B77F57DFAF73EC12DF2C51B6533B6CBF86B7C9126EA
E5333E2D0F10578CB41E3
2F6783AA74D910632710D383451D1E50A2E2503CD0432A5A882581BB65C8C8083C08D2EC7C7E
6FF9C2A33DC1D5D31C00C
F82E643659A037C421F3D71674232C8DEFFA1B9F78AEC497BE99D722A0B98083266DBA1C2576
```

Initialization with user defined keypair

```
FF169EF7FACDF9EF47C8C
BC3B63E8579800C65818D20AF1CC616559C559A634E92D6A2BD1080888A2262D4E921678C72A
F8B039385DDF9589879E6
03A199E96DE941446A0C5AF8251320A641F3BB74DB57CD055E85871BF0553B882CE3907A5F8C
F6436A874E643D84265E4
B92F2DD47C001048E1714C705C21";

int numKeys=1; //min numKeys=1, max numKeys=4
int useHash=1;

//For .NET4.0
e2eedplite.Init(numKeys, useHash, SymmetricAlgo.AES_256, RSA_KEY_N,
RSA_KEY_E, RSA_KEY_D);

//For .NET3.5
e2eedplite.Init(numKeys, useHash, SymmetricAlgo.AES_256,
    RSA_KEY_N, RSA_KEY_E, RSA_KEY_D, null, null, null, null, null, null,
null, null, null);
```

Initialization with specific public key index offset.

C#

```
int numKeys=1; //min numKeys=1, max numKeys=4
int useHash=1;
int pkiOffset = 888;

//For .NET4.0
e2eedplite.InitWithPkiOffset(numKeys, useHash,
SymmetricAlgo.AES_256, pkiOffset);

//For .NET3.5
e2eedplite.InitWithPkiOffset(numKeys, useHash,
SymmetricAlgo.AES_256, pkiOffset,
    null, null, null, null, null, null, null, null, null,
null, null, null);
```

Getting Data Protection Parameters

Get public key index (pki), random number (rn) and public key (pk).

The pki and rn should be stored in the web session of the application, while the rn and pk should be sent to the client for encryption. When the ciphertext is received from the client, the pki and rn should be retrieved from the client's corresponding web session and removed from web session.

Get DP params

Get DP params	
C#	
<pre>DPParams DPParam = e2eedplite.GetParams(); System.Console.WriteLine("pki = " + DPParam.pki.ToString() + ", rn = " + DPParam.randomNumber + ", pk = " + DPParam.pk);</pre>	
Decrypt the ciphertext	
C#	
<pre>byte[] plaintext = e2eedplite.Decrypt(ciphertext, DPParam.randomNumber, DPParam.pki); } System.Console.WriteLine("plaintext (" + plaintext.Length + " bytes) = " + Encoding.UTF8.GetString(plaintext));</pre>	
Shutdown the engine	
C#	
<pre>e2eedplite.Shutdown();</pre>	

E2EE Integration With PHP Application

E2EE DP Setup

The following is a sample setup assuming Apache HTTP server and PHP 5.5 is installed, and the OS is Linux.

1. Install "OpenSSL" library if it is not installed.
2. Place the php extension **php-e2eedplite.so** in the php extension folder.

```
$ sudo cp php-e2eedplite.so `php-config --extension-dir`
```

3. Place the e2eedplite shared library **libe2eedplite.so** in the */usr/lib/* or */usr/lib64/* folder (may vary with different linux distributions).

```
$ sudo cp libe2eedplite.so /usr/lib64
```

4. Add the following into */etc/php.ini*:

```
INI (.ini files)
```

```

extension=php-e2eedplite.so
;The number of keypairs to use (min=1, max=4).
e2eedplite.numKeys=1

;useHash=0 : input ciphertext does not contains a hash.
;useHash=1 : input ciphertext contains hash and hash
checking needs to be performed.
e2eedplite.useHash=1

;useSymmetric=0 : input ciphertext does not use
symmetric encryption for the plaintext (format 00).
;useSymmetric=1 : input ciphertext uses symmetric
encryption for the plaintext (format 02).
e2eedplite.useSymmetric=1

;symmetricKeyLength=128 : AES 128 is used for symmetric
encryption
;symmetricKeyLength=256 : AES 256 is used for symmetric
encryption
e2eedplite.symmetricKeyLength=256

```

5. Optionally, you can supply your own keys. For example:

INI (.ini files)

```

;RSA keypair #1 (Optional parameters)
e2eedplite.key1N=E3333C6ED2B525BA9528324A0F875C839EC09858668EAF3B13
2DC2F17B86BDBC330269F92EA21CFA
68BAE4EC7C87DBA27BAF8E1BC9E30386D996E96ED037F62EC908B1BD6FF800330E4
C7F8FD1EF051C64C5E398E77E6B754
E9C8016A659E4A01936DC2F94CC539C05E004E6D2210D782618E12F391E5ED2A978
24839E3D024E0AFE6090D8EBFB2A8F
BBCCE4398FF4CE3CB4DD1A537FFF10DEBE01F4E9969C0F3D340B6F340F795C88B41
A7595F1AF5D212660E610F88D39F2F
64DA8148283DF7446135A0B26E20D6E7D06E9938A5FB2458C10A21851C085E16B25
92766C6999D670D3B0912641F44D06
748D152B6AD12FF42D196B97BEA828239746045A1EC7
e2eedplite.key1E=010001
e2eedplite.key1D=3CF3CBECEB75E9DC696A9780462722021B77F57DFAF73EC12DF
2C51B6533B6CBF86B7C9126EAE5333
E2D0F10578CB41E32F6783AA74D910632710D383451D1E50A2E2503CD0432A5A882
581BB65C8C8083C08D2EC7C7E6FF9C
2A33DC1D5D31C00CF82E643659A037C421F3D71674232C8DEFFA1B9F78AEC497BE9
9D722A0B98083266DBA1C2576FF169
EF7FACDF9EF47C8CBC3B63E8579800C65818D20AF1CC616559C559A634E92D6A2BD
1080888A2262D4E921678C72AF8B03
9385DDF9589879E603A199E96DE941446A0C5AF8251320A641F3BB74DB57CD055E8
5871BF0553B882CE3907A5F8CF6436
A874E643D84265E4B92F2DD47C001048E1714C705C21

;RSA keypair #2 (Optional parameters)
;e2eedplite.key2N=
;e2eedplite.key2E=
;e2eedplite.key2D=

```

```
;RSA keypair #3 (Optional parameters)
;e2eedplite.key3N=
;e2eedplite.key3E=
;e2eedplite.key3D=

;RSA keypair #4 (Optional parameters)
;e2eedplite.key4N=
;e2eedplite.key4E=
;e2eedplite.key4D=
```

Usage

Check if an extension is loaded properly:

PHP

```
<?php
$loaded = extension_loaded('e2eedplite');
print "e2eedplite extension loaded = $loaded<br>";
?>
```

Get public key index (pki), random number (rn) and public key (pk). The pki and rn should be stored in the web session of the application, while the rn and pk should be sent to the client for encryption. When the ciphertext is received from the client, the pki and rn should be retrieved from the client's corresponding web session.

Get DP Params

PHP

```
<?php
$pki = "";
#Create a empty string of length 1000 for the public key
$pk = str_pad("", 1000);
#Create a empty string of length 100 for the random number
$rn = str_pad("", 100);

$ret = dpGetParams($pki, $pk, $rn);

print "The return value of dpGetParams(...) is $ret<br>";
print "    pki = $pki<br>";
print "    rn  = $rn<br>";
print "    pk  = $pk<br>";
?>
```

Decrypt the ciphertext

Decrypt the ciphertext

PHP

```
<?php
#Create a empty string of length same as the ciphertext
$plaintextLen = strlen($ciphertext);
$plaintext = str_pad("", $plaintextLen);

$ret =
dpDecrypt($plaintext,$plaintextLen,$pki,$rn,$ciphertext);
print "The return value of dpDecrypt(...) is $ret<br>";
print "    plaintextLen = $plaintextLen<br>";
print "    plaintext = $plaintext<br>";
?>
```

4. UAS E2EE Data Protection Lite Libraries

Client Library

Cipher parameters need to be provided to the client library to match the LiteDPClientConfig in the application server with a choice of:

- RSA encryption with hash
- RSA and AES encryption with hash

Javascript

The header of **amdp.js** file contains the Javascript API description:

Loading Javascript library	
JavaScript	
<pre><script language="javascript" src="amdp.js"></script> <script language="javascript" src="json2.js"></script></pre>	
Encryption	
JavaScript	
<pre>// Constant cipherparams: enable integrity checking using SHA2-256 var cipherparams_hash='{\"hash\":true,\"hashAlgo\": \"SHA- 256\"}'; var ciphertext = amdp.encrypt(cipherparams_hash,pk,ra,plaintext);</pre>	

Examples are provided in **e2eedp.war**:

- **simplelite-form.jsp** - Asks user data and encrypts it before submission.
- **simplelite-decrypt.jsp** - Decrypts the encrypted user data in the application server.
- **SimpleDataProtectionServlet.java** - Initializes E2EE DP (DataProtSvc).

Java / Android

The E2EE Data Protection Java client library works for both Android and Java applications.

Encryption

Java

```
// Returned by E2EE DP server-side SDK
String pk = ".....";
String rn = ".....";

// cipherparams should match the one configured at the server.
// Best is to get it from
DataProtClient.getBrowserCipherParams() (Java server-side SDK)
// or alternatively use these available constants:
// DataProtClient.CIPHERPARAMS_INTEGRITY_SHA_256
//
DataProtClient.CIPHERPARAMS_SYMMETRIC_AES_128_INTEGRITY_SHA_256
//
DataProtClient.CIPHERPARAMS_SYMMETRIC_AES_256_INTEGRITY_SHA_256
// DataProtClient.CIPHERPARAMS_SYMMETRIC_SM4_128_INTEGRITY_SM3
// or build using CipherParamsBuilder
// Symmetric encryption allows encryption of longer plaintext
String cipherparams =
DataProtClient.CIPHERPARAMS_SYMMETRIC_AES_128_INTEGRITY_SHA_256

String plaintext = "transaction data to be encrypted";
String ciphertext = DataProtClient.encrypt(cipherparams, pk,
rn, plaintext);
System.out.println("ciphertext: "+ciphertext)
```



Notes:

- SM cryptography support requires SM library
- AES-256 requires unlimited strength JCE policy

Android (deprecated/legacy)



Note: Use Java library for Android application use.

Encryption using RSA and SHA-256 hashing for integrity checking

Java

```
String plaintext = "12345678901234567890";
AxMxE2EEDpClient e2eeDp = new AxMxE2EEDpClient();
String res00 =
e2eeDp.encrypt(ParameterUtil.CIPHERPARAMS_HASH, plaintext,
pk, ra);
```

To support long data: Encryption using RSA and AES, and SHA-256 hashing for integrity checking

To support long data: Encryption using RSA and AES, and SHA-256 hashing for integrity checking

Java

```
String plaintText = "12345678901234567890";
AxMxE2EEDpClient e2eeDp = new AxMxE2EEDpClient();
String res02 =
e2eeDp.encrypt(ParameterUtil.CIPHERPARAMS_SYM_HASH,
plaintText, pk, ra);
```

ios

C/C++

```
//-----
/**
 *  encrypt method
 *
 *  @param format          the enum of CipherFormat, eg.
CIPHERPARAMS_INTEGRITY_SHA_256,CIPHERPARAMS_SYMMETRIC_AES_128_INTEGRITY_S
HA_256
 *  @param publicKey      NSString
 *  @param randomNumber   NSString
 *  @param plainText      NSString
 *
 *  @return cipher        NSString
 */
+ (NSString *)encrypt:(CipherFormat)format
                    pk:(NSString *)publicKey
                    rn:(NSString *)randomNumber
          plainText:(NSString *)plainText;
```

Encryption using RSA and SHA-256 hashing for integrity checking

C/C++

```
NSString *PlainText = @"12345678901234567890";
NSString *result = [E2eedpLite
encrypt:CIPHERPARAMS_INTEGRITY_SHA_256
                    pk:Pk
                    rn:Rn
          plainText:PlainText];
```

Java

```
//-----
/**
 *  encrypt method
```

```

*
* @param format          the enum of CipherFormat, eg.
CIPHERPARAMS_INTEGRITY_SHA_256,CIPHERPARAMS_SYMMETRIC_AES_128_INTEGRITY_S
HA_256
* @param publicKey      NSString
* @param randomNumber   NSString
* @param plainData      NSData
*
* @return cipher        NSString
*/
+ (NSString *)encrypt:(CipherFormat)format
                  pk:(NSString *)publicKey
                  rn:(NSString *)randomNumber
          plainData:(NSData *)PlainData;

```

To support long data: Encryption using RSA and AES, and SHA-256 hashing for integrity checking

C/C++

```
NSData *PlainData ; // to be initialized with long data
bytes
```

```
NSString *result = [E2eedpLite
encrypt:CIPHERPARAMS INTEGRITY SHA 256
```

$$\begin{array}{l} p_k : P_k \\ r_n : R_n \end{array}$$

```
plainData: PlainData];
```
